Welcome.
Let's build something that actually works.

You'll design and ship real, working AI agents in Python using the Claude Agent SDK — not just talk about theory.

Claude Agent SDK: Build AI Agents in Python

File Explorer Agent

AGENT 1 OF 3 · SECTION 2

Give it a folder. Get back a report.

INPUT

/project
  /src
  /tests
  README.md

your agent

Summary Report

3 Python files found
12 functions, 2 classes
Test coverage: partial
README: present

You give it a folder. It figures out the rest.

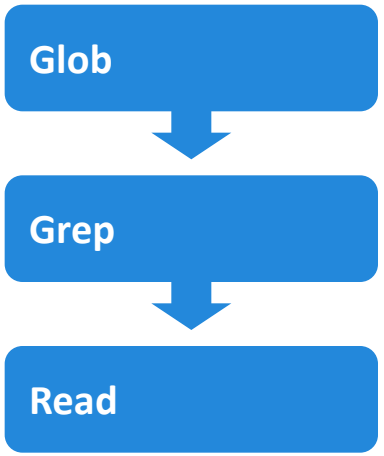
Codebase TODO Summarizer Agent

AGENT 2 OF 3 · SECTION 3

Multiple agents, working the same problem.

CODEBASE

```
auth.py   # TODO: add rate limit
api.py    # TODO: handle 429
db.py     # TODO: index users
tests.py  # TODO: add fixtures
```



search pipeline

TODO Report

4 TODOs found

HIGH · auth.py:14 · Add rate limiting
MED · db.py:87 · Index users table

Chain simple tools together — that's an agent.

Safe Code Review Agent

AGENT 3 OF 3 · SECTION 4

CODEBASE

auth.py

api.py

db.py

utils.py

READ ONLY

CODE REVIEW

auth.py
— SQL injection risk

→ parameterise queries

api.py
— no error handling

→ add try/except blocks

Cannot modify a single line of code.

What is the Claude Agent SDK?

Let's understand exactly what we're working with.

The Claude Agent SDK

THE DEFINITION

A Python library for building AI agents —

Programs that use Claude to **think**

and built-in tools to **act,**

running **autonomously** *until the task is done.*

Breaking Down the Definition

THE DEFINITION

Two concepts: Python library & AI agents

Python library

- You install it with pip.
- You import it in your code.
- You call it just like any other library.

AI agents

Not just answering questions.

Programs that take actions in the world — reading files, running commands, making decisions.

```
pip install claude-agent-sdk
```

That single command is all it takes to get started.

Breaking Down the Definition

THE DEFINITION

Three more concepts: think, act, and autonomously

Claude to think

Claude is the brain. It reads the situation, decides what to do next, and keeps reasoning until the task is complete.

Built-in tools to act

Read files, write files, run bash commands, search the web — no manual implementation. They come included.

Autonomously

You describe the task. The SDK figures out the steps. You don't need to be in the loop for every action.

The Core Value Proposition

Why not just call the Claude API directly?

If you call the Claude API directly,
you have to manage the loop yourself.

① Send a prompt to Claude
② Check if Claude wants to use a tool
③ Execute that tool yourself
④ Send the result back to Claude
⑤ Repeat — until Claude says it is done

You are in the loop every single step. You write all of that code.

Before vs. After

The same task — two very different experiences

WITHOUT Agent SDK

You
↓
Claude asks for a tool
↓
You execute tool manually
↓
Send result back to Claude
↓
(Repeat until finished)

Result

You're in the middle — every step.

VS

WITH Agent SDK

You describe the task
↓
Agent runs autonomously
↓
(SDK handles all tool loops internally)
↓
Result received

Result

You hand off the task and get back the result.

Comparison: Claude.ai vs. Claude Agent SDK

Two very different experiences built on the same underlying foundation

Claude.ai

The Conversational Interface

Human-Centric
Designed for direct interaction where you type and Claude responds.

Human-in-the-loop
Iterative process where you must provide input at every step.

Best For
Quick questions, creative brainstorming, and one-off tasks.

Claude Agent SDK

The Programmatic Engine

</>

Developer-Centric

Your code defines the goal and provides the tools.

Autonomous Execution
Claude plans and executes tool calls without human intervention.

Best For
Automating workflows, scaling agents, and complex product builds.

Comparison: Agent SDK vs. Anthropic Client SDK

Both are Python libraries. The difference is who manages the tool loop.

Anthropic Client SDK

Maximum control, maximum responsibility

→ You send messages to Claude

→ You implement tool execution

→ You manage the loop yourself

Use when: fine-grained control over every API call

Claude Agent SDK

Claude + built-in tools, loop handled for you

✅ Claude handles tool execution autonomously

✅ The loop is built in — no boilerplate

✅ You describe the task

Use when: building agents quickly, without writing the loop

*"Think of the Client SDK as the **raw ingredient**. The Agent SDK is the **finished kitchen** — with the appliances already installed."*

Comparison 3 — Agent SDK vs. Managed Agents

Same agent loop. Different deployment model.

	Agent SDK	Managed Agents
What it is	Python library	REST API service
Runs on	Your machine	Anthropic's servers
Filesystem access	Your filesystem, directly	Sandboxed environment
Infrastructure	You manage it	Anthropic manages it
Control	Full programmatic control	Control via JSON params

This course uses the Agent SDK — for full control, local filesystem access, and deep customisation.

Everything you need to build production agents is already here.

OUT OF THE BOX

Built-in Tools

Read · Write · Edit · Bash · Glob · Grep · WebSearch · WebFetch · AskUserQuestion

Hooks

Intercept and control agent behaviour at key lifecycle points — validate, log, block, transform

Sessions

Maintain context across multiple exchanges — resume or fork conversations

Permissions

Control exactly what the agent can and cannot do — read-only, auto-edit, custom approval

Subagents

Spawn specialised agents to handle focused subtasks in parallel

The Four Components

A breakdown of the architecture for building AI Agents in Python

Your Code

The “Starting Gun” that defines prompts and tool permissions.

The SDK

The “Relay” managing the agent loop and session persistence.

Claude

The “Reasoner” in the cloud that decides tool usage.

Claude Code

The “Executor” binary running tools locally on your machine.

Four Components — Not One Black Box

Most beginners think of the Agent SDK as a single thing. It isn't.

?

The “black box” assumption

- **Your Python Code**
- **The Agent SDK**
- **Claude — the LLM**
- **Claude Code — the binary**

Four distinct components — each with its own job

The Four Components at a Glance

Your Code

THE STARTING GUN

- Calls query()
- Sets tools & options
- Receives result

Agent SDK

THE RELAY

- Orchestrates all
- Manages loop
- Persists sessions

Claude (LLM)

THE REASONER

- Thinks & decides
- Never executes
- Cloud Native

Claude Code

THE EXECUTOR

- Runs tools locally
- Reads & edits
- Local Machine

The SDK is the only component that talks to all the others, acting as the central nervous system of your agent.

Component 1 — Your Python Code

THE STARTING GUN

Your code has exactly one job: fire the starting gun.

1 **Call query() with a prompt**

The instruction you give the agent

2 **Pass ClaudeAgentOptions**

Which tools are allowed, permissions

3 **Set up tool permissions**

Read-only? Auto-edit? You decide

4 **Receive the final result**

When the loop completes — you get the answer

Everything that happens between the starting gun and the final result is handled by the other three components.

Component 2 — The Agent SDK

THE RELAY — THE MIDDLEMAN

The SDK is the only component that talks to everyone else.

Launches Claude Code
Runs as a subprocess on your local machine

Calls Anthropic's API
Communicates with Claude (the LLM)

Translates Decisions
Turns tool decisions into shell commands

Feeds Results Back
Sends tool output to Claude for reasoning

Persists Conversations
Stores session state and history

Manages Agent Loop
Orchestrates until the task is complete

The SDK acts as the central nervous system, coordinating all interactions between the user, Claude, and the environment.

The Relay Race Analogy

Claude

The Brain

Decides the strategy

Agent SDK

The Relay Runner

Passes the baton

Claude Code

The Hands

Does the work

Without the SDK, Claude and Claude Code cannot talk to each other — the baton never gets passed.

Component 3 — Claude (the LLM)

THE REASONER · ☁ IN THE CLOUD

Claude is the brain. It lives in the cloud, accessed via Anthropic's API.

Context Acquisition

Receives prompt and full conversation context from the SDK

Reasoning Engine

Thinks and reasons about what should happen next

Tool Use Decision

Returns structured tool use decision — not execution

Iterative Evaluation

Looks at tool results and reasons for the next step

Task Completion

Decides when the task is complete and emits a final answer

Claude's Critical Distinction

✅

THINKING

- Receives context from the SDK
- Reasons about what to do
- Decides which tool to call
- Evaluates results
- Determines when task is done

❌

NOT DOING

- Does NOT read your files
- Does NOT run commands
- Does NOT touch your filesystem
- Does NOT execute tools
- Never acts — only decides

Claude does the THINKING, not the DOING. It decides — Claude Code executes.

Component 4 — Claude Code (the binary)

THE EXECUTOR · ON YOUR MACHINE

Claude Code is the hands. It runs locally as a native binary on your machine.

🚀

SDK Lifecycle

Launched by the Agent SDK as a subprocess when your agent starts

⚙️

Tool Execution

Actually executes tools — reads files, edits files, runs commands

➡️

Standard Input

Receives tool requests from the SDK via stdin as JSON

⬅️

Standard Output

Returns tool results to the SDK via stdout as JSON

🔒

Faithful Execution

Never makes decisions — it faithfully executes what it is told

Claude Code's Critical Distinction

✅

DOING

- Reads files from your filesystem
- Writes and edits files
- Runs bash commands & scripts
- Executes whatever tool is requested
- Returns results to the SDK

❌

NOT THINKING

- Does NOT reason about the task
- Does NOT decide what to do next
- Does NOT choose which tool to use
- Has no opinions or strategy
- Never initiates — only responds

Claude Code does the DOING, not the THINKING. It executes — Claude decides.

How the SDK and Claude Code Communicate

The SDK and Claude Code are constantly talking through **two pipes.**

Agent SDK

Writes JSON
tool requests

Reads JSON
results

PIPE 1 — stdin

Tool: Read | File: "auth.py"

PIPE 2 — stdout

Result: [contents of auth.py]

Claude Code

Reads JSON
tool requests (stdin)

Writes JSON
results (stdout)

Think of it as two walkie-talkies — one channel for sending instructions, one for sending results back. You don't need to know the JSON format. Just know that these pipes exist and that every tool request and result passes through them.

Sessions — The SDK Remembers Everything

The SDK persists every message, tool call, and result to disk.

.claude/sessions/session_id.jsonl

Pause & Resume

Stop an agent mid-task and pick it up later with full context intact

Full History

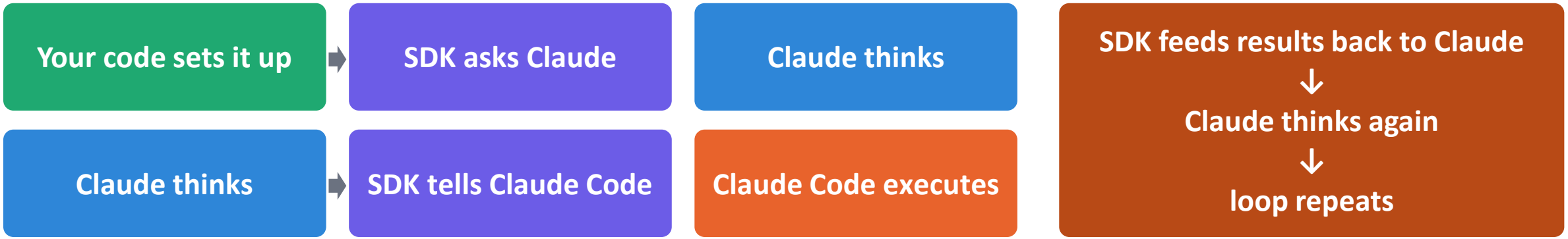
Claude references this history to make informed decisions at every step

Session Forking

Branch from any point to explore different approaches — you'll try this later

Sessions ensure that agents aren't just one-shot tools, but persistent entities with memory and continuity.

The One-Line Mental Model



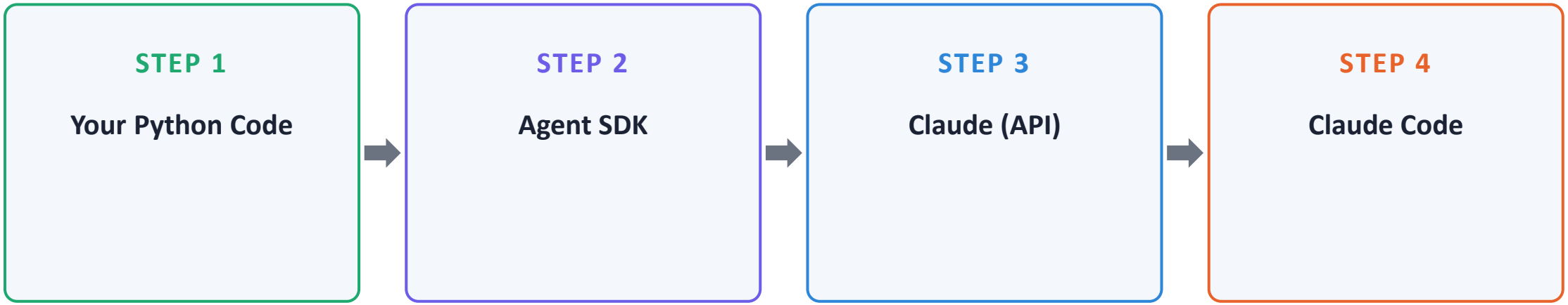
You don't need to memorise this. Just keep this picture in your head.

Every time you write agent code in this course — this is what is happening behind the scenes.

The next lecture will walk through each step of the loop in real time.

From Components to Motion

In the last lecture, you met the runners. **In this lecture, you watch the race.**



These four components pass work to each other in a cycle — that cycle is called the agent loop.

What Makes an Agent Autonomous?

An agent isn't a single request and response.

Single Call

send You send a prompt.

chat Claude answers.

check_circle Done.

Agent Loop

psychology Claude decides what to do.

settings_suggest It acts. It learns from results.

sync It decides again — until done.

*The loop is what turns Claude from a **chatbot** into an **agent.***

The Complete Agent Loop

Ten steps, three roles — trace how a single request flows through the loop.

1
RELAY
Your code calls SDK

2
RELAY
SDK sends to Claude

3
THINK
Claude decides tool

4
RELAY
SDK → JSON → stdin

5
DO
Claude Code executes

10
DO
Done or loop again

9
THINK
Claude reasons again

8
RELAY
SDK sends to Claude

7
RELAY
SDK reads stdout

6
DO
Result → stdout

THINK (Claude decides) DO (Claude Code executes) RELAY (SDK orchestrates)

The 4 Message Types

When you iterate over query(), four types of messages stream back.

TYPE 1

SystemMessage

Loop Step 2 — Session Initialised

Fires at the very start of every agent run. Contains session initialisation info including the session ID you'll use to resume later.

TYPE 2

AssistantMessage

Loop Step 9 — Claude Reasoning

Every Claude response arrives as an AssistantMessage. It carries a content list of blocks — TextBlock for text, ToolUseBlock when Claude calls a tool, ThinkingBlock for reasoning.

These two message types carry the **initialisation signal** and the **reasoning** of the agent.

How Tool Activity Arrives

Two more message types carry tool calls and their results.

TYPE 2 — continued

AssistantMessage + ToolUseBlock

Loop Step 3 — Claude's tool decision

When Claude calls a tool, an AssistantMessage arrives whose content includes a ToolUseBlock — with the tool name and input parameters.

TYPE 3

UserMessage + ToolResultBlock

Loop Step 7 — Tool execution result

When a tool finishes, a UserMessage arrives whose content includes a ToolResultBlock with the output.

No standalone ToolUseMessage or ToolResultMessage exist in the SDK.
Tool activity is always **a block inside an AssistantMessage or UserMessage.**

Message Type 4 — The One That Matters Most

The final message in every stream — your agent's answer.

TYPE 4

ResultMessage

Loop Step 10 — Task Complete

The loop has ended. Claude has produced its final answer.

The text in .result is identical to the last AssistantMessage TextBlock — the SDK copies it there. But ResultMessage is emitted by the framework, not Claude, and adds a **.result** field plus metadata: total_cost_usd, duration_ms, num_turns, is_error.

```
if hasattr(message, "result"): print(message.result)
```

Writing Your First query() Call

The Four Message Types

Tool activity is always a block inside a message — never a standalone message type.

SystemMessage
SDK generates
blocks: No content blocks

Session init. Fires before Claude sees your prompt. Carries session_id.

AssistantMessage
Claude decides
blocks: TextBlock, ToolUseBlock, ThinkingBlock

Every Claude response. Carries a content list — loop through message.content to read blocks.

UserMessage
SDK generates
blocks: ToolResultBlock

Tool results arrive here — not in another AssistantMessage. SDK feeding result back to Claude.

ResultMessage
SDK generates
blocks: No blocks — use .result

Loop complete. Final text + metadata: cost, turns, duration. Guaranteed last message.

ResultMessage vs. AssistantMessage + TextBlock

Same final text, two very different signals.

AssistantMessage + TextBlock	
Emitted by	Claude — the LLM
When	During the loop — possibly many times
Contains	Claude's in-progress text
Signal	Claude is speaking right now
Example	Commentary between two tool calls

ResultMessage	
Emitted by	The SDK framework — not Claude
When	Once, at the very end of the run
Contains	Same final text + metadata (cost, turns, duration)
Signal	Agent loop is fully complete
Example	hasattr(message, 'result') fires on this

The text is literally the same string. Use ResultMessage because it is the SDK's guarantee that the loop is done — and because it carries metadata no TextBlock has.

Build a File Explorer Agent

One Variable, One Place to Change

WHAT IT DOES

- Controls which Claude model every agent in the notebook uses
- Set once at the top — change it in one place to switch models
- For this course: claude-haiku-4-5 — fast and cost-efficient for learning
- Upgrade to claude-sonnet-4-5 or claude-opus-4-5 any time

```
# Change this to switch models

# e.g. "claude-sonnet-4-5"

MODEL_NAME = "claude-haiku-4-5"
```

Every ClaudeAgentOptions call in this notebook passes model=MODEL_NAME

The Colab Filesystem

Fresh VM every session

Colab runs on a Linux (Ubuntu) virtual machine. Every session starts with a completely fresh filesystem.

/content/ is home

Your working directory is /content/. Create all your files and folders here — it is the Colab equivalent of your home folder.

Files reset on disconnect

Everything under /content/ is wiped when the runtime resets. This is why we always create sample files programmatically.

The File Explorer Agent — How It Works

AGENT FLOW

1. You give it a goal — explore this project
2. Glob finds all files and folders
3. Read opens and reads each file
4. Claude understands and summarises each one
5. ResultMessage delivers the report to you

TOOLS

Glob

Find files by pattern — **/*.py, src/**/*.py

Read

Open and read any file in the working directory

ResultMessage

hasattr(message, 'result') — the final answer

The Prompt Is Your Primary Control Lever

Same tools. Same project. Completely different output.

PROMPT A — File Summaries

"Read each file. Write a one sentence summary of what it contains."

→ **A clean report — one sentence per file**

PROMPT B — Function Names

"Find all Python files. For each one, list all function names defined in it."

→ **A function inventory — every function in every .py file**

File Tools:
Read, Write, Edit

What We're Covering in This Lecture

Three tools for working with files — each suited to a different job.

Read

Opens any file and brings its full contents into the agent's context — Claude reasons about it, summarises it, extracts information from it.

Write

Creates brand new files based on your prompt. Critical: if the file already exists, Write overwrites it entirely.

Edit

Makes surgical, targeted changes to a specific part of an existing file — everything else is left exactly as it was.

Each tool demonstrated in isolation — allowed_tools locks the agent to one tool at a time.

Shell Tools:
Bash & Monitor

The Bash Tool

What it does

Executes any shell command from within the agent — terminal commands, scripts, git operations, system checks, package installs.

The agent decides which commands to run. You describe the goal in plain English.

When to use it

- Checking system information
- Running build or shell scripts
- Git operations
- Any task a developer would do in a terminal

⚠ Safety Note

Bash is the most powerful tool in the Claude Agent SDK.

It can run any command on the system — including destructive ones.

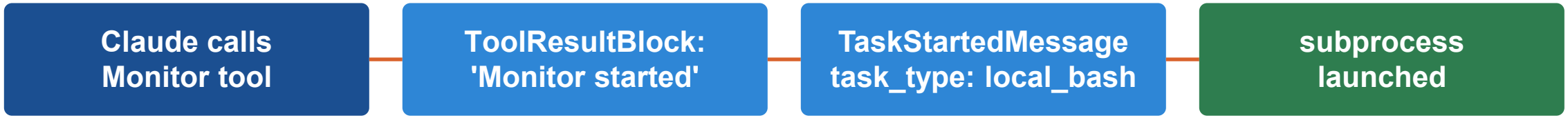
Be deliberate about every task you give an agent with Bash access.

Section 4 — Permissions & Safety — covers this in full.

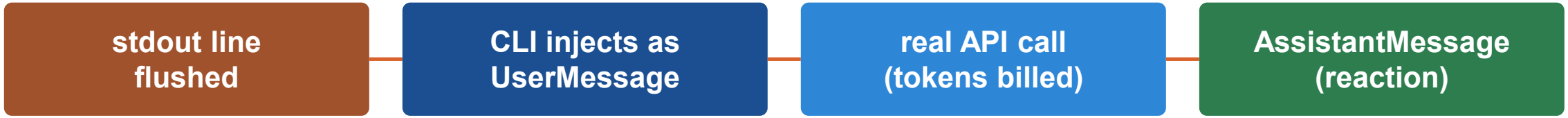
How Monitor Actually Works

Bash runs a command, waits, and returns full output at the end — Monitor runs a script in the background and injects each stdout line as a fresh message

TOP ROW



PER STDOUT LINE (EVERY 3 SECONDS)



Those injected UserMessages are never surfaced to your Python loop — just like your initial prompt.
The SDK gives you outputs (AssistantMessage), not the inputs it manages on your behalf.
Exception: ToolResultBlocks in UserMessages ARE surfaced — they close a tool call Claude explicitly made.

Two Things That Make or Break Monitor

Get either of these wrong and Monitor silently stops being real-time

`flush=True`

Without it:
Python buffers stdout. The OS delivers all 5 lines to Monitor at once when the process exits. Claude sees a batch, not a stream.

With it:
Each print() flushes immediately to stdout. Monitor delivers one event per line. Claude reacts genuinely in real time.

```
print(step, flush=True)
```

Bash in allowed_tools

Monitor has no shell of its own. It needs Bash as its execution engine to launch the subprocess.

Confirmed by TaskStartedMessage:

```
task_type: "local_bash"
```

Without Bash:
Claude reads the script file first, predicts the output, then runs it. Reactions are based on prediction — not genuine real-time observation.

Always: `allowed_tools=["Monitor", "Bash"]`

The Full Message Taxonomy for a Monitor Run

Message / Block	Lives In	When It Arrives	Surfaced to Python?
`ThinkingBlock`	AssistantMessage	Claude reasoning before acting	Yes
`ToolUseBlock`	AssistantMessage	Claude calls Monitor tool	Yes
`ToolResultBlock`	UserMessage	System confirms Monitor started	Yes — closes tool call
`TaskStartedMessage`	SystemMessage	Background task goes live (task_type: local_bash)	Yes
`TextBlock (per line)`	AssistantMessage	Claude reacts to each stdout line	Yes — arrives progressively
`Injected UserMessage`	UserMessage	CLI injects each stdout line into context	No — SDK hides it
`TaskNotificationMessage`	SystemMessage	Subprocess finishes	Yes
`ResultMessage`	—	Closes each turn (one per turn, not total)	Yes — use last_result

Search Tools: Glob & Grep

What We Cover in This Lecture

Glob

Finds files by name and extension pattern across an entire directory tree.

Example patterns:

`**/*.py` · `src/**/*.py` · `tests/*.py`

Grep

Searches inside files for content matching a text or regex pattern.

Example patterns:

`TODO` · `^def` · `\d+`

Regex — Just Enough to Be Useful

Pattern	What it matches	Example use
`TODO`	The exact string "TODO"	Find all TODO comments
`^def`	Lines starting with def	Find all function definitions
`.*`	Any characters (wildcard)	Match anything
`\d+`	One or more digits	Find numbers in files

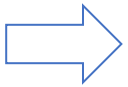
WebSearch & WebFetch

A New Boundary: The Agent Goes Live

LECTURES 3.1 – 3.3: LOCAL AGENT

- Read, Write, Edit
- Bash, Monitor
- Glob, Grep

All local — your machine only



LECTURE 3.4: INTERNET AGENT

- WebSearch
- WebFetch
- Combined research workflow

Live internet — the world

Two New Tools

`WebSearch`

Performs a live web search and returns relevant results—titles, snippets, and URLs. Claude synthesizes these into a coherent answer.

Use when:

- Finding current information on a topic
- Discovering relevant URLs to fetch later

`WebFetch`

Retrieves the full content of a specific URL and parses it into readable text that Claude can reason about.

Use when:

- You already have a specific URL
- You need its complete content, not just a snippet

The Combined Research Workflow

WebSearch	WebFetch	Synthesis
Discovers relevant URLs from a query	Reads the full content of the best URL	Claude synthesises a coherent answer

The same search-then-retrieve pattern as Glob + Grep — but pointed at the entire internet.

One Important Note Before We Code

Live output will vary — and that is the point.

Because these tools hit the live web, the output will look different every time you run a cell.

What to evaluate:

- Is the structure of the response what we asked for?
- Is the synthesis coherent and relevant?
- Did the agent find the right information?

Not identical text. Not specific version numbers.

Normal Flow vs Human-in-the-Loop

NORMAL FLOW	HUMAN-IN-THE-LOOP FLOW
1 Send prompt (plain string)	1 Send prompt (async generator)
2 Claude chooses tools	2 Claude chooses tools
3 SDK executes tools automatically	3 Claude calls AskUserQuestion
4 Claude processes results	4 ⏸ Execution pauses for user input
5 Claude returns final answer	5 User answers → Claude continues
6 Code receives ResultMessage	6 Code receives ResultMessage

VS

Hooks vs Callbacks — A Critical Distinction

can_use_tool (Callback)	PreToolUse dummy hook (Hook)
Role: Makes allow / deny decisions	**Role:** Observes and signals — no decisions
Pauses execution? YES — Claude waits for return value	**Pauses execution?** NO — does not block Claude
Where pause happens: Inside callback, when input() is called	**What it signals:** Keep-alive: HTTP connection stays open
Return value: PermissionResultAllow or Deny	**Return value:** {"continue_": True}
Fires: For every tool call — routes by tool_name	**Fires:** Before each tool use (hook fires first)

The Complete Human-in-the-Loop Flow — 15 Steps

PHASE 1: REQUEST & PREPARATION

01 call query() with async generator prompt

02 prompt flows through open HTTP connection

03 Claude starts working on the task

04 Claude calls AskUserQuestion internally

05 dummy hook fires → keep-alive signal sent

06 can_use_tool callback fires, receives tool_name + data

07 code checks: tool_name == "AskUserQuestion"

08 displays question and options to user

PHASE 2: INTERACTION & COMPLETION

09 PAUSE — input() called, user types

10 user types answer, hits Enter

11 code maps answer to option label

12 returns PermissionResultAllow with ToolResponse

13 ToolResponse flows through open connection

14 Claude processes answers and continues task

15 Claude returns final answer — ResultMessage

The Keep-Alive Mechanism

Your Application	HTTP Connection (Stays Open)	Claude
query() called	→ Prompt flows in	receives prompt
streaming async generator	keep-alive heartbeat	starts task
dummy hook returns	*Sent via dummy hook*	calls AskUserQuestion
continue_: True	⏸ Waiting for user input...	waits for ToolResponse
can_use_tool callback fires	← ToolResponse flows back	receives answers
input() pauses execution	← ResultMessage flows back	finishes task
PermissionResultAllow returned		

Streaming input *opens the interactive channel. The* ***dummy hook*** *keeps it alive. The* ***callback*** *uses it to send the ToolResponse back.*

Permission Modes

Detailed Flow Logic & Decision Matrix

Six gates evaluate every tool call, in a fixed order, until one returns a final decision

1 PreToolUse Hooks

"deny": **Stop & DENY**

"defer": Resume later

"allow"/"ask": **To Gate 2**

2 disallowed_tools

If present → **DENY (always)**

3 allowed_tools

If present → **APPROVE (always)**

4 Permission Mode

bypass: **APPROVE**

acceptEdits: **APPROVE (files)**

plan: **DENY (write tools)**

5 canUseTool

If Registered: **YOUR DECISION**

Note: skipped if mode is dontAsk → DENY

6 No Callback

dontAsk: **DENY**

default: Blocked in non-interactive sessions

GATE SUMMARY

- Gates 1–3: Global logic.
- Gate 4: Configured modes.
- Gates 5–6: UI/Interactive logic.

First gate to reach a final decision (APPROVE/DENY) stops the entire flow.

allowed_tools vs disallowed_tools

Two configuration lists that behave very differently — and fire at different gates

GATE 2

disallowed_tools

Absolute deny list

- Listed tools: **DENY — no exceptions, ever**
- Holds in ALL modes — including *bypassPermissions*

Even if a tool is in allowed_tools, Gate 2 fires first. The deny wins.

GATE 3

allowed_tools

Pre-approval list

- Listed tools: **APPROVE immediately at Gate 3**
- Unlisted tools: fall through to Gate 4 — **NOT blocked**

Common misconception: this is NOT a whitelist. Unlisted tools are not denied.

! **Gate order matters:** Gate 2 (deny) fires before Gate 3 (allow). So a tool in both lists is always denied — the deny rule wins.

The Five Permission Modes (Python SDK)

Permission modes only apply at Gate 4 — for tools in neither allowed_tools nor disallowed_tools

Mode	What it does	When to use
default	No auto-approvals — unmatched tools trigger canUseTool callback	General purpose — most common starting point
dontAsk	Anything not pre-approved is denied — canUseTool never called	Headless agents with a fixed, explicit tool surface
acceptEdits	Auto-approves file edits and filesystem operations	When you trust Claude's edits and want faster iteration
bypassPermissions	All tools approved at Gate 4 — use with extreme caution	Controlled environments only — Claude has full system access
plan	Read-only — Claude plans but does not edit files	Code review, proposing changes before executing them

The bypassPermissions Gotcha

⚠ **Critical safety point — understand this before using bypassPermissions in production.**

GATE 3

allowed_tools

allowed_tools=["Read"]

- "Read" is approved and stops here.
- Bash, Write, Edit… fall through.
- They are **NOT** blocked by this gate.

GATE 4

bypassPermissions

mode="bypassPermissions"

- **ALL** tools approved automatically.
- Bash? **APPROVED.**
- Write? Edit? **APPROVED.**

✓ **The Fix: Use disallowed_tools at Gate 2**

disallowed_tools=["Bash"], mode="bypassPermissions"

Gate 2 is the only gate that remains active in bypassPermissions mode. It denies Bash before Gate 4 can approve it.

Read-Only Agent vs Auto-Edit Agents

What We'll Build — Three Configurations

Read-Only Agent

CONFIGURATION

allowed_tools:

Read, Glob, Grep

permission_mode:
dontAsk

Report only — files untouched

Auto-Edit Combo 1

CONFIGURATION

allowed_tools:

Read, Write, Edit, Glob, Grep

permission_mode:
dontAsk

Adds docstrings — Gate 3 approves all tools

Auto-Edit Combo 2

CONFIGURATION

allowed_tools:

Read, Glob, Grep

permission_mode:
acceptEdits

Adds docstrings — Gate 4 approves Write/Edit

The Permission Spectrum

This lecture builds both extremes so you understand the full range before designing agents in the middle.

OBSERVE ONLY

CONFIGURATION

allowed_tools: Read, Glob, Grep

permission_mode: dontAsk

READ + WRITE + EDIT

CONFIGURATION

allowed_tools: Read, Write, Edit, Glob, Grep

permission_mode: dontAsk OR acceptEdits

Most production agents live here

READ-ONLY

Max Safety

Zero Autonomy

AUTO-EDIT

Max Autonomy

Use with care

Three Configurations — Gate Paths Compared

Combo 1 and Combo 2 produce identical agent behaviour through different gate paths

	Read-Only Agent	Auto-Edit Combo 1	Auto-Edit Combo 2
allowed_tools	Read, Glob, Grep	Read, Write, Edit, Glob, Grep	Read, Glob, Grep
permission_mode	dontAsk	dontAsk	acceptEdits
Read tools approved at	Gate 3	Gate 3	Gate 3
Write/Edit approved at	Never — denied	Gate 3	Gate 4
Can modify files?	✗	✓	✓
Best for	Auditing, review, analysis	Explicit whitelist pattern	Mode-based write access

Blocking Dangerous Operations

The Problem with Broad Permissions

Two extremes fail in opposite ways — the fix is a surgical, rule-based middle ground.

✗ Two extremes — neither works

disallowed_tools

Blocks the entire tool. Agent loses all Bash capabilities.

allowed_tools = ['Bash']

Agent has full Bash. One bad prompt could be catastrophic — rm -rf, sudo, curl | bash.

✓ The surgical approach — hooks

Allow Bash. Register a PreToolUse hook.

The hook intercepts every Bash call and checks the command against a list of dangerous patterns.

- **Safe commands:** Pass through to execution.
-
- **Dangerous commands:** Deny, with a reason so Claude can find a safer alternative.

can_use_tool vs Hook Callbacks

Two mechanisms for controlling tool execution — one asks a human, the other applies rules.

	can_use_tool	Hook Callbacks
Purpose	Interactive — pause & ask a human	Automated rules — no human needed
Streaming Input?	Required (Python)	Not required
Can Block Tool?	Yes	Yes — PreToolUse with deny
Can Modify Input?	Yes	Yes — updatedInput
Can Ask Human?	Yes — designed for this	No — automated only
Best For	AskUserQuestion, user approvals	Blocking, modification, logging

Available Hook Events — Python

Nine lifecycle events fire as an agent runs — this lecture uses PreToolUse.

HOOK EVENT	WHEN IT FIRES
PreToolUse	**Just before Claude executes any tool ← we use this one**
PostToolUse	Just after a tool finishes executing
PostToolUseFailure	When a tool execution fails
UserPromptSubmit	When a user prompt is submitted
Stop	When the agent stops execution
SubagentStart	When a subagent is spawned
SubagentStop	When a subagent finishes
PreCompact	Before conversation compaction
PermissionRequest	When a permission dialog would show
Notification	When the agent sends a status message

Three Surgical Blocking Patterns

Each pattern intercepts a call at PreToolUse and returns a decision — deny, or allow with a rewrite.

1 Block Dangerous Bash Commands

Regex-scan every Bash call. Match rm -rf, sudo, chmod 777, curl pipes, /etc/ writes.

Return: deny + reason

2 Block Writes to Sensitive Paths

Intercept every Write and Edit call. Block /etc/, /usr/, ~/.ssh/, shell dotfiles, /root/.

Return: deny + reason

3 Allow with Modifications

Silently redirect writes outside /content/blocking_demo/ into the safe directory. Claude thinks it succeeded.

Return: allow + updatedInput